

Development guide

1 Environment configuration

- 1.1 Add SDK to your project
- 1.2 Declare permissions
- 1.3 Declaration component
- 1.4 Obfuscation rules

2 Usage

- 2.1 Create the CRPBleClient
- 2.2 Scan BLE device
- 2.3 Connect
- 2.4 Time
- 2.5 Firmware
- 2.6 Battery
- 2.7 User Info
- 2.8 Weather
- 2.9 Steps
- 2.10 Sleep
- 2.11 Unit system
- 2.12 Quick view
- 2.13 Goals
- 2.14 Watch face
- 2.15 Alarm
- 2.16 Watch version
- 2.17 Language
- 2.18 Notification
- 2.19 Sedentary reminder
- 2.20 Find the watch
- 2.21 Heart rate
- 2.22 Blood pressure
- 2.23 Blood oxygen
- 2.24 Take a photo
- 2.25 Mobile phone related operations
- 2.26 RSSI
- 2.27 Shut down
- 2.28 Do not disturb
- 2.29 Breathing light
- 2.30 ECG
- 2.31 Physiological cycle
- 2.32 Find phone
- 2.35 Music player
- 2.36 Drink water reminder
- 2.37 Heart rate alarm
- 2.38 Training
- 2.39 Protocol version
- 2.40 Temperature
- 2.41 Display time
- 2.42 Hand washing reminder
- 2.43 Sets local city
- 2.44 Temperature system
- 2.45 Brightness
- 2.46 Classic Bluetooth binding and unbinding
- 2.47 Contacts
- 2.48 Battery Saving
- 2.49 Pill Reminder
- 2.50 Tap to wake
- 2.51 Create Bond
- 2.52 HRV
- 2.53 Stress
- 2.54 E-Card
- 2.55 Calendar Event
- 2.56 GPS
- 2.58 Vibration Strength
- 2.59 Custom Key
- 2.60 World Clock
- 2.61 Stock
- 2.62 AI Chat
- 2.62 Muslim

3 Change log

- 1.8.2
- 1.7.1
- 1.4.2
- 1.3.7
- 1.3.4
- 1.3.3
- 1.3.2
- 1.3.1
- 1.3.0
- 1.2.10
- 1.2.9
- 1.2.8
- 1.2.7
- 1.2.6

1.2.5
1.2.4
1.2.3

Development guide

1 Environment configuration

1.1 Add SDK to your project

Copy crpblelib-*.aar to the project's libs directory. If there is an old version of the aar package in it, please delete it.

1.2 Declare permissions

Configure permissions in AndroidManifest.xml

```
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

<!-- Android 12 Add permission-->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
```

1.3 Declaration component

Please declare the following components in the application tag

```
<activity android:name="com.crrepa.ble.trans.upgrade.NotificationActivity"/>
<service android:name="com.crrepa.ble.trans.upgrade.DfuService"/>
<service
    android:name="com.realsil.sdk.dfu.DfuService"
    android:enabled="true"
    android:exported="false" />
<service
    android:name="com.example.dfulibrary.SifliDfuService"
    android:enabled="true"
    android:exported="false" />
```

1.4 Obfuscation rules

Perform the following configuration when generating apk for code obfuscation

```
-keep class com.crrepa.ble.** { *; }
```

2 Usage

The following contains all the functions supported by the MoYoung Watch, and a certain watch only supports part of the functions. To find out if your watch supports a feature, contact us.

2.1 Create the CRPBleClient

CRPBleClient is the entry point of the SDK. The client needs to maintain an instance of CRPBleClient. It is recommended to initialize CRPBleClient in Application.onCreate().

```
CRPBleClient bleClient = CRPBleClient.create(context);
```

2.2 Scan BLE device

1. Start scan

Normal scanning can be started with permission enabled and Bluetooth enabled. When the watch is found during scanning, it is called back by CRPScanCallback.onScanning(). After the scan is finished, all the watches found during the scan are recalled by CRPScanCallback.onScanComplete(). The scan duration can be set in milliseconds. Because the Bluetooth scan operation is time-consuming, the recommended scan time is 10 seconds.

```
bleClient.scanDevice(scanCallback, 10 * 1000);
```

2. Cancel scan

Stop scanning while scanning, and canceling scanning also triggers CRPScanCallback.onScanComplete().

```
bleClient.cancelScan();
```

2.3 Connect

1. Connect

Get the Mac Address to the watch by scanning the CRPScanDevice of the callback. Connect to the watch via CRPBleDevice.connect() and callback state via CRPBleConnectionStateListener.onConnectionStateChange(). It is recommended to increase the appropriate delay when the reconnection is disconnected, so that the system can recover resources and ensure the connection success rate.

```
String address = CRPScanDevice.getDevice().getAddress();
CRPBleDevice bleDevice = bleClient.getBleDevice(address);
CRPBleConnection bleConnection = bleDevice.connect();
bleConnection.setConnectionStateListener(bleConnectionStateListener);
```

2. Reconnect

There are two ways to reconnect:

- New GATT connection (recommend)

```
// close GATT
CRPBleConnection.close();
// delay 3 seconds
String address = CRPScanDevice.getDevice().getAddress();
CRPBleDevice bleDevice = bleClient.getBleDevice(address);
CRPBleConnection bleConnection = bleDevice.connect();
bleConnection.setConnectionStateListener(bleConnectionStateListener);
```

- Continue to connect

```
CRPBleConnection.connect();
```

2.4 Time

1. Sync Time

Keep the watch and the phone in the same time.

```
bleConnection.syncTime();
```

2. Set Time System

Sets the time system of the watch.

```
bleConnection.sendTimeSystem(byte type);
```

CRPTimeSystemType:

TIME_SYSTEM_12	TIME_SYSTEM_24
12-Hour Time	24-Hour Time

3. Get Time System

Query the time system of the watch.

```
bleConnection.queryTimeSystem(CRPDeviceTimeSystemCallback callback);
```

2.5 Firmware

1. Query firmware version

Query the current firmware version of the watch.

```
bleConnection.queryFirmwareVersion(CRPDeviceFirmwareVersionCallback);
```

2. Check firmware

Version is the current firmware version. The new firmware information is called back via CRPDeviceNewFirmwareVersionCallback.onNewFirmwareVersion().

```
bleConnection.checkFirmwareVersion(CRPDeviceNewFirmwareVersionCallback, version, CRPFirmwareUpgradeType);
```

CRPFirmwareUpgradeType:

NORMAL_UPGRADE_TYPE	BETA_UPGRADE_TYPE	FORCED_UPDATE_TYPE
Normal upgrade. Used for regular upgrades.	BETA upgrade. Used for internal testing version upgrades.	Forced upgrade. Generally not used, for major updates, forcing users to upgrade.

CRPFirmwareVersionInfo:

version	changeNotes	changeNotesEn	type	mcu
Current firmware version number of the watch	change note	english change note	Upgrade type, same as CRPFirmwareUpgradeType	MCU type, used to distinguish upgrade mode

3. Firmware upgrade

The chip firmware upgrade method of each platform is different, and the upgrade method of each platform is as follows.

o Nordic/Goodix

1. Start upgrade

The upgrade progress is called back through CRPBleFirmwareUpgradeListener.

```
bleConnection.startFirmwareUpgrade(isGoodix, CRPBleFirmwareUpgradeListener);
```

2. Abort upgrade

After the upgrade succeeds or fails, this interface can be called to exit the firmware upgrade process.

```
bleConnection.abortFirmwareUpgrade();
```

3. Query DFU status

Check whether the watch is in the DFU data transmission state. In the new firmware, the watch will restart multiple times during the DFU process. When the watch is in the DFU state, avoid sending other commands to the watch. The result is called back via CRPDeviceDfuStatusCallback.onDeviceDfuStatus().

```
bleConnection.queryDeviceDfuStatus(CRPDeviceDfuStatusCallback);
```

CRPDeviceDfuStatusCallback

DEVICE_STATUS_NORMAL	DEVICE_STATUS_DFU
Normal	DFU

o Huntersun

1. Query Huntersun dfu address

The upgrade mode of Huntersun watch is different from the normal mode. It has a separate broadcast name and Mac address, and the Mac address is called back through CRPDeviceDfuAddressCallback. Before the upgrade, you need to query the Mac address of the firmware upgrade mode of the watch. You can save this Mac address and re-upgrade after the upgrade fails.

```
bleConnection.queryHsDfuAddress(CRPDeviceDfuAddressCallback);
```

2. Enable Huntersun Dfu mode

After the watch enters the firmware upgrade mode, it will disconnect the Bluetooth connection from the normal mode, and initiate a firmware upgrade after the Bluetooth connection is disconnected. After the watch enters the firmware upgrade mode, it is not allowed to initiate a normal Bluetooth connection, which will cause the firmware upgrade to fail.

```
bleConnection.enableHsDfu();
```

3. Start upgrade

The upgrade progress is called back through CRPBleFirmwareUpgradeListener. After the upgrade is successful, the watch will restart and return to the normal mode; after the upgrade fails, the firmware upgrade can be restarted.

```
HsDfuController hsDfuController = new HsDfuController();
hsDfuController.setUpgradeListener(CRPBleFirmwareUpgradeListener);
hsDfuController.setAddress(address);
hsDfuController.start();
```

4. Abort upgrade

After the upgrade succeeds or fails, this interface can be called to exit the firmware upgrade process.

```
hsDfuController.abort();
```

o Realtek

1. Start upgrade

The upgrade progress is called back through CRPBleFirmwareUpgradeListener. Please add in AndroidManifest:

```
<service
    android:name="com.realsil.sdk.dfu.DfuService"
    android:enabled="true"
    android:exported="false" />
```

```
RtkDfuController controller = new RtkDfuController();
controller.setUpgradeListener(new CRPBleFirmwareUpgradeListener());
controller.start(String address);
```

2. Abort upgrade

```
controller.abort();
```

o Sifli

1. Start upgrade

Unable to abort upgrade. The upgrade progress is called back through CRPBleFirmwareUpgradeListener. Please add in AndroidManifest:

```
<service
    android:name="com.example.dfulibrary.SifliDfuService"
    android:enabled="true"
    android:exported="false" />
```

```
SifliDfuController controller = new SifliDfuController();
controller.setUpgradeListener(new CRPBleFirmwareUpgradeListener());
controller.start(String address);
```

o JIELI

1. Start upgrade

The upgrade progress is called back through CRPBleFirmwareUpgradeListener.

```
JieliDfuController controller = new JieliDfuController();
controller.setUpgradeListener(new CRPBleFirmwareUpgradeListener());
controller.start();
```

2. Abort upgrade

```
controller.abort();
```

2.6 Battery

1. Query the current battery of the watch. When the battery level of the watch exceeds 100, it means the watch is charging.

```
bleConnection.queryDeviceBattery();
```

2. Set up the watch battery listener.

```
bleConnection.setDeviceBatteryListener(CRPDeviceBatteryListener);
```

3. Subscription battery

When the battery of the watch changes, it will be called back through CRPDeviceBatteryListener.onDeviceBattery(battery).

```
bleConnection.subscribeDeviceBattery();
```

2.7 User Info

1. Set user information

Set the user's personal information to the watch.

```
bleConnection.sendUserInfo(CRPUserInfo);
```

CRPUserInfo:

weight	height	gender	age
Weight (used to calculate calories)	Height (used to calculate the distance of the activity)	Gender (used to measure blood pressure or blood oxygen)	Age (for measuring blood pressure or blood oxygen)

2. Set steps length

In the watch firmware 1.6.6 and above, you can set the step length to the watch to calculate the activity data more accurately.

```
bleConnection.sendStepLength(length);
```

2.8 Weather

1. Today's weather

Set the weather today to the watch.

```
bleConnection.sendTodayWeather(CRPTodayWeatherInfo);
```

CRPTodayWeatherInfo:

city	lunar	festival	pm25	temp	weatherId
City	Lunar Festival (not necessary)	festival(not necessary)	PM2.5	Real-time temperature	Weather status

CRPWeatherId:

CLOUDY	FOGGY	OVERCAST	RAINY	SNOWY	SUNNY	SANDSTORM	HAZE
partly cloudy	fog	overcast	rainy	snowy	sunny	sandstorm	haze

2. Weather in the next 7 days

Set the weather for the next 7 days to the watch.

```
bleConnection.sendFutureWeather(CRPFutureWeatherInfo);
```

CRPFutureWeatherInfo.FutureBean:

weatherId	lowTemperature	highTemperature
Weather status	lowest temperature	Maximum temperature

3. Weather listener

The watch can save real-time weather for 2 hours, and the weather information will be cleared after 2 hours. When the watch has no weather information today, when the watch switches to the weather interface, the weather is reset by `CRPWeatherChangeListener.onUpdateWeather()`.

```
bleConnection.setWeatherChangeListener(CRPWeatherChangeListener);
```

2.9 Steps

1. Set steps listeners

All activity-related data is called back through the `CRPStepChangeListener`.

```
bleConnection.setStepChangeListener(CRPStepChangeListener);
```

2. Query today's steps

Get the number of steps today, and the result is called back by `CRPActionChangeListener.onStepChange()`.

```
bleConnection.syncStep();
```

CRPStepInfo:

steps	distance	calories	time
steps	Distance (in meters)	Calories (units of kilocalories)	Activity duration, (the default value is 0, which means the watch does not support)

3. Query history steps

The watch can save the number of active steps in the last three days, and can query the number of active steps in a given day. The result is called back via `CRPStepChangeListener.onHistoryStepChange()`.

```
// Sifli watches support the history of the last 7 days, while other watches only support the last three days. It is recommended to query the next one after querying the callback in the previous day.
bleConnection.queryHistoryStep(CRPHistoryDay);
```

4. Set steps category listener

The step count data is called back via the `CRPStepsCategoryChangeListener`.

```
bleConnection.setStepsCategoryListener(CRPStepsCategoryChangeListener);
```

5. Query steps category

Some watches support the classification statistics for the last two days. The result is called back by `CRPStepsCategoryChangeListener.onStepsCategoryChange()`.

```
bleConnection.queryStepsCategory(CRPStepsCategoryDateType);
```

2.10 Sleep

1. Set sleep listener

All sleep related data is called back via CRPSleepChangeListener.

```
bleConnection.setSleepChangeListener(CRPSleepChangeListener);
```

2. Query today's sleep

Get today's sleep data, the result is called back by CRPSleepChangeListener.onSleepChange(). The watch sleep clear time is 8:00 pm, and the watch records the sleep time period from 8:00 pm to 10:00 am the next day.

```
bleConnection.syncSleep();
```

CRPSleepInfo:

totalTime	restfulTime	lightTime	soberTime	SLEEP_STATE_RESTFUL	SLEEP_STATE_LIGHT	SLEEP_STATE_SOBER	SLEEP_STATE_REM	remTime
Total sleep time	restful time	light time	awake time	restful	light sleep	awake	rem sleep	rem sleep time

CRPSleepInfo.DetailBean:

startTime	endTime	totalTime	type
start time	end time	total time	sleep state

3. Query history sleep

The watch can save sleep data for the last three days and can query the sleep data for a given day. The result is called back by CRPSleepChangeListener.onPastSleepChange().

```
bleConnection.queryHistorySleep(CRPHistoryDay);
```

4. Query history nap

Some watches support detecting nap records. The result is called back through CRPSleepChangeListener.onHistoryNapSleepChange().

```
bleConnection.queryHistoryNapSleep(CRPHistoryDay historyDay)
```

2.11 Unit system

1. Set the unit system

```
bleConnection.sendMetricSystem(CRPMetricSystemType);
```

CRPMetricSystemType:

METRIC_SYSTEM	IMPERIAL_SYSTEM
METRIC	IMPERIAL

2. Query the unit system

```
bleConnection.queryMetricSystem(CRPDeviceMetricSystemCallback callback);
```

2.12 Quick view

1. Set the quick view state

Turns the quick view on or off.

```
bleConnection.sendQuickView(state);
```

2. Query the quick view state

The result is called back via CRPDeviceQuickViewCallback.onQuickView().

```
bleConnection.queryQuickView(CRPDeviceQuickViewCallback);
```

3. Set the effective time for quick view

The watch supports the effective period of quick view, and the ban bright screen is effective only when the watch is illuminated.

```
bleConnection.sendQuickViewTime(CRPPeriodTimeInfo);
```

4. Get the effective time for quick view

Check the effective time period of quick view set by the watch. The result is called back by CRPDevicePeriodTimeCallback.onPeriodTime().

```
bleConnection.queryQuickViewTime(CRPDevicePeriodTimeCallback);
```

2.13 Goals

1. Set goal steps

Push the user's target step number to the watch. When the number of active steps reaches the target step, the watch will have a target to reach the reminder.

```
bleConnection.sendGoalSteps(steps)
```

2. Get goal steps

Query the target number of steps in the watch, and the result is called back by CRPDeviceGoalStepCallback.onGoalStep().

```
bleConnection.queryGoalStep(CRPDeviceGoalStepCallback);
```

3. Set daily goals

The daily goal is to set a goal value for each day.

```
bleConnection.sendDailyGoals(CRPDailyGoalsInfo);
```

CRPDailyGoalsInfo:

steps	calories	trainingTime	distance
The goal steps	The goal calories (unit: Kcal)	The goal distance time (unit: minute)	The goal distance

4. Query daily goals

Callback via CRPDailyGoalsCallback.onDailyGoals().

```
bleConnection.queryDailyGoals(CRPDailyGoalsCallback);
```

5. Set training day goals

The training day is to identify a certain day as a training day, and the goal value can be set separately, and the goal value of the training day will override the daily goal value.

```
bleConnection.sendTrainingDayGoals(CRPDailyGoalsInfo);
```

6. Query training day goals

Callback via CRPTrainingDayGoalsCallback.onTrainingDayGoals().

```
bleConnection.queryTrainingDayGoals(CRPTrainingDayGoalsCallback);
```

7. Set training day

```
bleConnection.sendTrainingDays(CRPTrainingDayInfo);
```

CRPTrainingDayInfo:

enable	trainingDays
ture is enable; otherwise vice versa	The training day, same alarm

8. Query training day

Callback via CRPTrainingDayGoalsCallback.onTrainingDays().

```
bleConnection.queryTrainingDay(CRPTrainingDayGoalsCallback);
```

2.14 Watch face

1. Set display watch face index

The watch supports a variety of different watch face, which can be switched freely.

```
// The index of the built-in watch face starts from 1 and increases sequentially. The index of the download watchface is after the built-in watch face.
bleConnection.sendDisplayWatchFace(byte index);
```

2. Query display watch face index

Query the index of the watch face being used by the watch, and the result is called back through CRPDeviceWatchFacesCallback.onDisplayWatchFace().

```
bleConnection.queryDisplayWatchFace(CRPDeviceDisplayWatchFaceCallback);
```


3. Query the watch face layout

Query the watch watchface layout information, and the result is called back by `CRPDeviceWatchFaceLayoutCallback.onWatchFaceLayoutChange()`.

```
bleConnection.queryWatchFaceLayout(CRPDeviceWatchFaceLayoutCallback);
```

`CRPWatchFaceLayoutInfo`:

timePosition	timeTopContent	timeBottomContent	textColor	backgroundPictureMd5	height	width	thumbHeight	thumbWidth	compressionType
time position	content displayed above time	content displayed below time	font color(RGB)	The background image MD5 has a length of 32 bits. When padded with 0, the background image restores the default background.	The watch face height, default 240 px.	The watch face width, default 240 px.	The thumb watch face height, The default is 0, which means it is not supported	The thumb watch face width, The default is 0, which means it is not supported	The compression type(LZO, RGB_DEDUPLICATION, RGB_LINE, SIFL, ORIGINAL)

`CRPWatchFaceLayoutType`:

WATCH_FACE_TIME_TOP	Time is at the top right
WATCH_FACE_TIME_BOTTOM	Time is at the bottom right
WATCH_FACE_CONTENT_CLOSE	Do not display anything
WATCH_FACE_CONTENT_DATE	Date
WATCH_FACE_CONTENT_SLEEP	Sleep
WATCH_FACE_CONTENT_HEART_RATE	Heart Rate
WATCH_FACE_CONTENT_STEP	Steps

4. Set the watchface layout

```
bleConnection.sendWatchFaceLayout(CRPWatchFaceLayoutInfo);
```

5. Set watch face background

The watchface of the 1.3-inch color screen supports the replacement of the background image with a picture size of 240 * 240 px. Compressed indicates whether the picture needs to be compressed (the watch with the master control of 52840 does not support compression and is fixed to false); timeout indicates the timeout period, in seconds. The replacement progress is called back through the `CRPWatchFaceSwitchListener`.

```
bleConnection.sendWatchFaceBackground(CRPWatchFaceBackgroundInfo, CRPFileTransListener)
```

`CRPWatchFaceBackgroundInfo`:

bitmap	thumbBitmap	type	timeout
The bitmap of background image	The bitmap of thumbnail	<code>CRPWatchFaceLayoutInfo.CompressionType</code>	Transmission timeout(unit: <code>TimeUnit.SECONDS</code>). 30 seconds by default.

6. Abort watch face background

Abort transfer during watch face background image transfer.

```
bleConnection.abortWatchFaceBackground();
```

7. Get support watch face type

Some watches add a new watch face to query the supported watch face types. The result is called back by `CRPDeviceSupportWatchFaceCallback`.

```
bleConnection.querySupportWatchFace(CRPDeviceSupportWatchFaceCallback);
```

`CRPSupportWatchFaceInfo`:

displayWatchFace	supportWatchFaceList
The watchface id	The support watchface type list, used to get the watch faces available on the server

8. Get the watch face store

According to the watch face type supported by the watch, obtain a list of watch faces that the watch can be replaced. The result is called back via `CRPDeviceWatchFaceStoreCallback.onWatchFaceStoreChange()`.

```
bleConnection.queryWatchFaceStore(List<Integer> list, String firmwareVersion, int pageCount, int pageIndex, CRPDeviceWatchFaceStoreCallback callback);
```

Get the list of available watch faces by way of paging query.

9. Get the watch face information of the watch face Id

Gets the watchface information of the watchface ID, and the result is called back by `CRPDeviceWatchFaceCallback.onWatchFaceChange()`.

```
bleConnection.queryWatchFaceOfID(id, CRPDeviceWatchFaceCallback);
```

10. Send download watch face

Send the watchface file of the new watchface to the watch, during which the watch will restart. The transfer progress is called back by CRPWatchFaceTransListener.

```
bleConnection.sendWatchFace(CRPWatchFaceInfo info, CRPFileTransListener listener, timeout);
```

CRPWatchFaceInfo:

file	type
The watch face file	WacthFaceType

11. Abort Download watch face

```
bleConnection.abortWatchFace();
```

12. Delete Download watch face

The Sifli watch supports deleting the watch face, and the deletion result is called back through CRPWatchFaceDeleteCallback.

```
bleConnection.deleteWatchFace(int id, CRPWatchFaceDeleteCallback callback);
```

13. Query the available storage space of the watch

Query the available storage space of the watch to determine whether the watch can download new dials. Via the CRPAvailableStorageCallback.onAvailableSize() callback.

```
bleConnection.queryAvailableStorage(CRPAvailableStorageCallback);
```

14. Query JIELI watchface information

Query JIELI watch watchface information and use it when obtaining available dials in the watchface store. Through CRPJieliWatchFaceCallback.onWatchFaceInfo() callback.

```
bleConnection.queryJieliWatchFaceInfo(CRPJieliWatchFaceCallback);
```

15. Send watchface ID

After JIELI Watch sends the watch face, it needs to send the watch face ID.

```
// id is CRPWatchFaceDetailsInfo.id
bleConnection.sendWatchFaceId(int id);
```

16. New watchface store

1. Query supported watch face types

Through CRPWatchFaceStoreTagCallback.onWatchFaceInfo() callback.

```
bleConnection.queryWatchFaceStoreTagList(CRPWatchFaceStoreRequestInfo info,
                                          CRPWatchFaceStoreTagCallback callback);
```

CRPWatchFaceStoreRequestInfo:

storeType	typeList	firmwareVersion	perPageCount	pageIndex	maxSize	apiVersion	feature
CRPWatchFaceStoreType	List of watch face types supported by the watch	The current firmware version of the watch	Pagination query; quantity per page	Pagination query; page number	Available storage space on your watch(unit: byte)	JIELI watch exclusive: CRPJieliWatchFaceInfo.apiVersion	JIELI watch exclusive:CRPJieliWatchFaceInfo.feature

2. Query all watch faces of watch face type

Through CRPWatchFaceStoreCallback.onWatchFaceStoreChange() callback.

```
// tagId is CRPWatchFaceStoreTagInfo.tagId
bleConnection.queryWatchFaceStoreList(CRPWatchFaceStoreRequestInfo info, int tagId,
                                       CRPWatchFaceStoreCallback callback);
```

3. Query watch face details

Through CRPWatchFaceDetailsCallback.onNewWatchFaceChange() callback.

```
bleConnection.queryWatchFaceDetail(CRPWatchFaceDetailsRequestInfo info, CRPWatchFaceDetailsCallback callback);
```

CRPWatchFaceDetailsRequestInfo:

storeType	id	firmwareVersion	maxSize	apiVersion	feature
CRPWatchFaceStoreType	The watchface ID	The current firmware version of the watch	Available storage space on your watch(unit: byte)	JIELI watch exclusive: CRPJieliWatchFaceInfo.apiVersion	JIELI watch exclusive:CRPJieliWatchFaceInfo.feature

2.15 Alarm

Some watches support the new version of the alarm clock (up to 8 alarm clocks, which can be deleted), and other watches only support the old version of the alarm clock (3 fixed alarm clocks, which cannot be deleted).

1. Set alarm(old version)

The watch supports three alarm clocks, and the alarm information can be set separately according to the alarm number. A single alarm supports setting the date.

```
bleConnection.sendAlarm(CRPAlarmInfo)
```

CRPAlarmInfo:

id	hour	minute	repeatMode	enable
The alarm id	hour (24-hour format)	minute	Repeat mode(For example, repeat on Monday and Tuesday, repeatMode = MONDAY + TUESDAY)	enable

Alarm id:

FIRST_CLOCK	SECOND_CLOCK	THIRD_CLOCK
First alarm clock	Second alarm clock	Third alarm clock

Repeat mode:

SINGLE	SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY	EVERYDAY
Single, only valid today.	Repeat on Sunday	Repeat on Monday	Repeat on Tuesday	Repeat on Wednesday	Repeat on Thursday	Repeat on Friday	Repeat on Saturday	Everyday

2. Query all alarm(old version)

Query the alarm information saved by the watch, and the result is called back by CRPAlarmCallback.onAlarmList().

```
bleConnection.queryAllAlarm(CRPAlarmCallback);
```

3. Set alarm(new version)

The alarm index cannot exceed 8.

```
bleConnection.sendNewAlarm(CRPAlarmInfo info);
```

4. Delete alarm(new version)

```
// Delete the alarm with the specified index
bleConnection.deleteNewAlarm(int id);
```

5. Delete all alarm(new version)

```
bleConnection.deleteAllNewAlarm();
```

6. Query all alarm(new version)

Callback via CRPAlarmCallback.onNewAlarmList().

```
bleConnection.queryAllNewAlarm(CRPAlarmCallback callback);
```

2.16 Watch version

1. Set the watch version

The watch is divided into Chinese and international versions. The Chinese version only supports Simplified Chinese. The international version supports multiple languages and traditional Chinese.

```
bleConnection.sendDeviceVersion(CRPDeviceVersionType);
```

CRPDeviceVersionType:

CHINESE_EDITION	INTERNATIONAL_EDITION
Chinese Edition	International Edition

2. Get the watch version

Query the version that the watch is using, and the result is called back by `CRPDeviceVersionCallback.onDeviceVersion()`.

```
bleConnection.queryDeviceVersion(CRPDeviceVersionCallback);
```

2.17 Language

1. Set the watch language

Set the language of the watch.

```
bleConnection.sendDeviceLanguage(CRPDeviceLanguageType);
```

`CRPDeviceLanguageType`:

LANGUAGE_ENGLISH	LANGUAGE_CHINESE	LANGUAGE_JAPANESE	LANGUAGE_KOREAN	LANGUAGE_GERMAN	LANGUAGE_FRENCH	LANGUAGE_SPANISH	LANGUAGE_ARABIC	LANGUAGE_RUSSIAN	LANGUAGE_TRAI
English	Chinese Simplified	Japanese	Korean	German	French	Spanish	Arabic	Russian	traditional Chinese

Note: Italian and Portuguese only support watch firmware 1.7.1 and above.

2. Get the watch language

Query the language that the watch is using and the list of languages supported by the watch. The result is called back by `CRPDeviceLanguageCallback.onDeviceLanguage(int type, int[] languageArray)`.

type	languageArray
The type of language being used	The List of all language types supported by the watch

```
bleConnection.queryDeviceLanguage(CRPDeviceLanguageCallback);
```

2.18 Notification

1. Set other message

Turn on or off other message.

```
bleConnection.sendOtherMessageState(state);
```

2. Get other message

Query other message enable status, and the result is called back by `CRPDeviceOtherMessageCallback.onOtherMessage()`.

```
bleConnection.queryOtherMessageState(CRPDeviceOtherMessageCallback);
```

3. Send message

Send various types of message content to the watch.

```
bleConnection.sendMessage(CRPMessageInfo);
```

`CRPMessageInfo`:

message	type	versionCode	isHs	isSmallScreen
Message content	Message type(<code>CRPBleMessageType</code>)	Firmware version (for example: MOY-AA2-1.7.6, which is 176)	Whether the MCU is HS, please confirm the MCU type with us.	Is the watch screen smaller than 1 inch

`CRPBleMessageType`:

MESSAGE_PHONE	phone
MESSAGE_SMS	SMS
MESSAGE_WECHAT	WeChat (Chinese Edition)
MESSAGE_QQ	QQ
MESSAGE_FACEBOOK	FACEBOOK
MESSAGE_TWITTER	TWITTER
MESSAGE_WHATSAPP	WHATSAPP
MESSAGE_WECHAT_IN	WeChat (International Edition)
MESSAGE_INSTAGREM	INSTAGREM
MESSAGE_SKYPE	SKYPE
MESSAGE_KAKAOTALK	KAKAOTALK
MESSAGE_LINE	LINE
MESSAGE_OTHER	Other

4. End call

When the watch receives a phone type message push, the watch will vibrate for a fixed time. Call this interface to stop the watch vibration when the watch is connected or the phone is hung up.

```
bleConnection.sendCallOffHook();
```

5. Set call number listener

Watches that support calls can make calls on the watch, and the outgoing phone number can be obtained through `CRPCallNumberListener.onCallNumber()`.

```
bleConnection.setCallNumberListener(CRPCallNumberListener);
```

6. Set call contact name

Send the name of the outgoing contact, and the incoming contact still uses `sendMessage`.

```
bleConnection.sendCallContactName(name);
```

7. Query the list of messages supported by the watch

Some watches have a built-in list of supported App messages. Callback via `CRPMessageListCallback.onMessageList()`.

```
// See CRPBleMessageType for the message type
bleConnection.queryMessageList(CRPMessageListCallback);
```

2.19 Sedentary reminder

1. Set sedentary reminder state

Turn the sedentary reminder on or off.

```
bleConnection.sendSedentaryReminder(state);
```

2. Query sedentary reminder state

Query the sedentary reminder status, and the result is called back by `CRPDeviceSedentaryReminderCallback.onSedentaryReminder()`.

```
bleConnection.querySedentaryReminder(CRPDeviceSedentaryReminderCallback);
```

3. Set sedentary reminder time

Set a valid period for sedentary reminders.

```
bleConnection.sendSedentaryReminderPeriod(CRPSedentaryReminderPeriodInfo);
```

CRPRemindersToMovePeriodInfo:

period	steps	startHour	endHour
Sedentary reminder period (unit: minute)	Maximum number of steps	Start time (24-hour clock)	End time (24-hour clock)

4. Get sedentary reminder time

Query the watch for sedentary reminder valid period, the result is called back by `CRPDeviceRemindersToMovePeriodCallback.onSedentaryReminderPeriod()`.

```
bleConnection.querySedentaryReminderPeriod(CRPDeviceSedentaryReminderPeriodCallback);
```

2.20 Find the watch

Look for the watch and the watch will vibrate for a few seconds after receiving this command.

```
bleConnection.findDevice();
```

2.21 Heart rate

1. Set heart rate listener

All heart rate related data will be called back via CRPHeartRateChangeListener.

```
bleConnection.setHeartRateChangeListener(CRPHeartRateChangeListener);
```

2. Query last dynamic heart rate measurement

The dynamic heart rate is measured in an unconnected state and the watch can save the last measurement. The result is called back by CRPHeartRateChangeListener.onMeasureComplete().

```
bleConnection.queryLastDynamicRate(CRPHistoryDynamicRateType);
```

CRPHeartRateInfo:

startMeasureTime	measureData	timeInterval	heartRateType
Start time (in milliseconds)	Heart rate measurements, a single data representing the average 2 heart rate over the measurement interval.	Time interval of heart rate measurement data (in minutes)	Heart rate data type

HeartRateType:

PART_HEART_RATE	TODAY_HEART_RATE	YESTERDAY_HEART_RATE
Action heart rate	Today heart rate	Yesterday heart rate

3. Enable timing to measure heart rate

The watch supports 24-hour timed measurement of heart rate, starting from 0:00, you can set the measurement interval, the time interval is a multiple of 5 minutes.

```
// The measurement interval = interval * 5 minutes
bleConnection.enableTimingMeasureHeartRate(interval);
```

4. Disable timing to measure heart rate

Turn off the timing to measure the heart rate.

```
bleConnection.disableTimingMeasureHeartRate();
```

5. Query timing to measure heart rate status

The query timing measures the heart rate on state, and the result is called back by CRPDeviceTimingMeasureHeartRateCallback.onTimingMeasure().

```
bleConnection.queryTimingMeasureHeartRate(CRPDeviceTimingMeasureHeartRateCallback);
```

6. Query today's heart rate measurement data

Today's heart rate measurement is divided into two types, which are obtained according to the measurement method supported by the corresponding watch. The result is called back by CRPHeartRateChangeListener.on24HourMeasureResult().

```
bleConnection.queryTodayHeartRate(CRPHeartRateType);
```

CRPHeartRateType:

TIMING_MEASURE_HEART_RATE	ALL_DAY_HEART_RATE
Timed heart rate measurement	24-hour continuous measurement

7. Query history heart rate

Query the heart rate data of the previous day, and the result is called back by CRPHeartRateChangeListener.on24HourMeasureResult().

```
bleConnection.queryHistoryTimingHeartRate(CRPHistoryDay)
```

8. Query Action data

Some watches support heart rate measurement in a variety of motion modes. The measurements include other motion-related data such as heart rate and calories. This interface is used to obtain data such as calories. The watch can save the last three sports data. Supporting 24-hour continuous measurement of the watch, the exercise heart rate can be obtained from the 24-hour heart rate data according to the movement up time; other watch exercise heart rate and dynamic heart rate acquisition methods are consistent.

```
bleConnection.queryMovementHeartRate();
```

CRPMovementHeartRateInfo:

type	startTime	endTime	validTime	steps	distance	calories
Sport mode	Start time (unit: milliseconds)	End time (unit: milliseconds)	Effective duration of exercise (unit: second)	Number of steps (partial motion mode is not supported)	Active distance (partial motion mode is not supported)	Calories

Sport mode:

WALK_TYPE	RUN_TYPE	BIKING_TYPE	ROPE_TYPE	BADMINTON_TYPE	BASKETBALL_TYPE	FOOTBALL_TYPE	SWIM_TYPE
Walking	Run	bicycle	rope skipping	badminton	basketball	football	Swim

9. Start measure once heart rate

Start measuring a single heart rate and the result is called back by CRPHeartRateChangeListener.onOnceMeasureComplete().

```
bleConnection.startMeasureOnceHeartRate();
```

10. Stop measure once heart rate

End a once measurement. A measurement time that is too short will result in no measurement data.

```
bleConnection.stopMeasureOnceHeartRate();
```

11. Query history once heart rate

Call back through CRPHeartRateChangeListener.onHistoryHeartRate(List list);

```
bleConnection.queryHistoryHeartRate();
```

2.22 Blood pressure

1. Set blood pressure listener

All blood pressure related data will be recalled via CRPBloodPressureChangeListener.

```
bleConnection.setBloodPressureChangeListener(CRPBloodPressureChangeListener);
```

2. Start measure blood pressure

Start measuring blood pressure.

```
bleConnection.startMeasureBloodPressure();
```

3. Stop measure blood pressure

Stop measuring blood pressure, too short a measurement time will result in no measurement results. The measurement results are called back via CRPBloodPressureChangeListener.onBloodPressureChange().

```
bleConnection.stopMeasureBloodPressure();
```

4. Enable continue blood pressure

```
bleConnection.enableContinueBloodPressure();
```

5. Disable continue blood pressure

```
bleConnection.disableContinueBloodPressure();
```

6. Query continue blood pressure state

Call back through CRPBloodPressureChangeListener.onContinueState(boolean state).

```
bleConnection.queryContinueBloodPressureState();
```

7. Query last 24 hour blood pressure

Call back through CRPBloodPressureChangeListener.onContinueBloodPressure(CRPBloodPressureInfo info);

```
bleConnection.queryLast24HourBloodPressure();
```

8. Query history once blood pressure

Call back through CRPBloodPressureChangeListener.onHistoryBloodPressure();

```
bleConnection.queryHistoryBloodPressure();
```

2.23 Blood oxygen

1. Set blood oxygen listener

All blood oxygen related data will be called back via CRPBloodOxygenChangeListener.

```
bleConnection.setBloodOxygenChangeListener(CRPBloodOxygenChangeListener);
```

2. Start Measure blood oxygen

Start measuring blood oxygen.

```
bleConnection.startMeasureBloodOxygen();
```

3. Stop measure blood oxygen

Stop measuring blood oxygen, too short a measurement time will result in no measurement results. The result is called back by CRPBloodOxygenChangeListener.onBloodOxygenChange().

```
bleConnection.stopMeasureBloodOxygen();
```

4. Enable timing measure blood oxygen

```
// The measurement interval = interval * 5 minutes
bleConnection.enableTimingMeasureBloodOxygen(int interval);
```

5. Disable timing measure blood oxygen

```
bleConnection.disableTimingMeasureBloodOxygen();
```

6. Query timing measure blood oxygen state

Call back through CRPBloodOxygenChangeListener.onTimingMeasure(int interval);

```
bleConnection.bleConnection.queryTimingBloodOxygenMeasureState();
```

7. Query timing blood oxygen

Call back through CRPBloodOxygenChangeListener.onContinueBloodOxygen(CRPBloodOxygenInfo info);

```
bleConnection.queryTimingBloodOxygen(CRPBloodOxygenTimeType timeType);
```

8. Enable continue blood oxygen

```
bleConnection.enableContinueBloodOxygen();
```

9. Disable continue blood oxygen

```
bleConnection.disableContinueBloodOxygen();
```

10. Query continue blood oxygen state

Call back through CRPBloodOxygenChangeListener.onContinueState(boolean state);

```
bleConnection.queryContinueBloodOxygenState();
```

11. Query last 24 hour blood oxygen

Call back through CRPBloodOxygenChangeListener.onContinueBloodOxygen(CRPBloodOxygenInfo info);

```
bleConnection.queryLast24HourBloodOxygen();
```

12. Query history once blood oxygen

Call back through CRPBloodOxygenChangeListener.onHistoryBloodOxygen();

```
bleConnection.queryHistoryBloodOxygen();
```

2.24 Take a photo

1. Enable camera view

```
bleConnection.enterCameraView();
```


2. Exit camera view

```
bleConnection.exitCameraView();
```

3. Set photo monitor

Long press the watch photo interface, you can trigger the camera's camera command, callback through `CRPCameraOperationListener.onTakePhoto()`.

```
bleConnection.setCameraOperationListener(CRPCameraOperationListener);
```

2.25 Mobile phone related operations

The watch can issue commands such as music control and hang up calls, and callbacks via `CRPPhoneOperationListener.onOperationChange()`.

```
bleConnection.setPhoneOperationListener(CRPPhoneOperationListener);
```

CRPPhoneOperationType:

MUSIC_PLAY_OR_PAUSE	MUSIC_PREVIOUS	MUSIC_NEXT	REJECT_INCOMING	VOLUME_UP	VOLUME_DOWN	MUSIC_PLAY	MUSIC_PAUSE
Play / Pause	Previous	Next	Hang up the phone. You can press and hold the trigger on the call alert interface.	Turn up the volume	Turn down the volume	Play	Pause

2.26 RSSI

1. Set RSSI listening

The watch RSSI is read and the result is called back by `CRPDeviceRssiListener.onDeviceRssi()`.

```
bleConnection.setDeviceRssiListener(CRPDeviceRssiListener);
```

2. Read the watch RSSI

Read the real-time RSSI value of the watch.

```
bleConnection.readDeviceRssi();
```

2.27 Shut down

The watch is turned off.

```
bleConnection.shutdown();
```

2.28 Do not disturb

1. Set the do not disturb time

The watch supports the Do Not Disturb period. Do not display message push and sedentary reminders during the time.

```
bleConnection.sendDoNotDisturbTime(CRPPeriodTimeInfo);
```

CRPPeriodTimeInfo:

startHour	startMinute	endHour	endMinute
Start time hours (24-hour clock)	Start time minutes	End time hours (24-hour clock)	End time minutes

2. Get the do not disturb time

Check the do not disturb time set by the watch. The result is called back by `CRPDevicePeriodTimeCallback.onPeriodTime()`.

```
bleConnection.queryDoNotDisturbTime(CRPDevicePeriodTimeCallback);
```

CRPDevicePeriodTimeCallback:

DO_NOT_DISTURB_TYPE	QUICK_VIEW_TYPE
Do not disturb	Quick view

2.29 Breathing light

1. Set the breathing light

Some watches support breathing lights and turn the breathing lights on or off.

```
bleConnection.sendBreathingLight(enable);
```

2. Get the status of the breathing light

Check the watch breathing light on. The result is called back via CRPDeviceBreathingLightCallback.onBreathingLight().

```
bleConnection.queryBreathingLight(CRPDeviceBreathingLightCallback);
```

2.30 ECG

1. Set ECG listener

All ECG related data will be recalled via CRPBleECGChangeListener.

```
bleConnection.setECGChangeListener(CRPBleECGChangeListener);
```

2. Measuring ECG

Start measuring ECG. The ECG measurement time is 30S, and the user needs to touch the left and right electrodes of the watch with both hands. ECG measurement data is retraced during CRPBleECGChangeListener.onECGChange().

```
bleConnection.startECGMeasure();
```

3. Stop measuring ECG

Used to actively stop ECG measurement during ECG measurement.

```
bleConnection.stopECGMeasure();
```

4. Detect new ECG measurement methods

In the new measurement mode, the watch can save the last unsent measurement result; the old version does not.

```
boolean newMeasurementVersion = bleConnection.isNewECGMeasurementVersion();
```

5. Get the last ECG data

Query the ECG data saved by the watch, and the result is called back by CRPBleECGChangeListener.onECGChange().

```
bleConnection.queryLastMeasureECGData();
```

6. Send heart rate during ECG measurement

Using the data obtained by the measurement, the instantaneous heart rate is calculated by the ECG algorithm library and sent to the watch.

```
bleConnection.sendECGHeartRate(heartRate);
```

2.31 Physiological cycle

1. Set the physiological cycle reminder

Some watch support the female physiological cycle reminder, set or turn off the physiological cycle reminder.

```
bleConnection.sendPhysiologicalPeriod(CRPPhysiologicalPeriodInfo);
```

CRPPhysiologicalPeriodInfo:

physiologicalPeriod	menstrualPeriod	startDate	menstrualReminder	ovulationReminder	ovulationDayReminder	ovulationEndReminder	reminderHour	reminderMinute
Physiological cycle (unit: day)	Menstrual period (unit: day)	Start date of this physiological cycle	Menstruation begins to remind (the day before the menstrual period reminder)	Ovulation reminder (reminder the day before ovulation)	Ovulation Day Reminder (Reminder on the day before ovulation)	Reminder at the end of the ovulation period (a reminder one day before the end of the ovulation day)	Reminder time (hours, 24 hours)	Reminder time (minutes)

2. Query physiological cycle reminder

Query the reminder information of the physiological cycle that has been saved in the watch. The result is called back via CRPDevicePhysiologicalPeriodCallback.

```
bleConnection.queryPhysiologicalPeriod(CRPDevicePhysiologicalPeriodCallback);
```

2.32 Find phone

1. Set find phone listener

```
bleConnection.setFindPhoneListener(CRPFindPhoneListener);
```

2. Start find phone

After receiving the callback to find the mobile phone for the bracelet, the APP can vibrate and play the ringtone reminder, and you can reply to the bracelet by this command. Some bracelets support this command.

```
bleConnection.startFindPhone();
```

3. End finding phone

When the user retrieves the phone, the vibrating and ringing reminder ends, and the bracelet can be returned by this command. Some bracelets support this command.

```
bleConnection.stopFindPhone();
```

2.35 Music player

1. Set player state

Some bracelets can show that the music player is playing or paused.

```
bleConnection.setMusicPlayerState(CRPMusicPlayerStateType);
```

CRPMusicPlayerStateType:

MUSIC_PLAYER_PAUSE	MUSIC_PLAYER_PLAY
pause	play

2. Set song name

Some bracelets can display information about the song being played.

```
bleConnection.sendSongTitle(title);
```

3. Set lyrics

```
bleConnection.sendLyrics(String lyrics);
```

4. Close Music Control

```
bleConnection.closeMusicControl();
```

5. Set max volume

```
bleConnection.sendCurrentVolume(int volume);
```

6. Set Current volume

```
bleConnection.sendMaxVolume(int volume);
```

2.36 Drink water reminder

1. Set drinking reminder

Set information such as the start time of drinking reminders.

```
bleConnection.sendDrinkWaterReminder(CRPDrinkWaterPeriodInfo);
```

CRPDrinkWaterPeriodInfo:

enable	startHour	startMinute	count	period	currentCups
enable	hours	minutes	reminders	reminder interval	the current water intake

2. Query drinking reminder

Query the current drinking reminder information of the bracelet. The result is called back through `CRPDeviceDrinkWaterPeriodCallback.onDrinkWaterPeriod(CRPDrinkWaterPeriodInfo)`.

```
bleConnection.queryDrinkWaterReminderPeriod(CRPDeviceDrinkWaterPeriodCallback);
```

2.37 Heart rate alarm

1. Set the heart rate alarm value

Sets the heart rate alarm status and alarm heart rate value.

```
bleConnection.setMaxHeartRate(heartRate, enable);
```

2. Query the heart rate alarm value

Query the status of the bracelet heart rate alarm and the heart rate alarm value. The result is called back by `CRPDeviceMaxHeartRateCallback.onHeartRate(heartRate, enable)`.

```
bleConnection.queryMaxHeartRate(CRPDeviceMaxHeartRateCallback);
```

2.38 Training

1. Start training

Some bracelets support instructions to turn on the training mode. type is the same as CRPMovementHeartRateInfo.type.

```
bleConnection.startMovement(type);
```

2. Sets training state

Training is divided into three states, pause, continue and end.

```
bleConnection.setMovementState(CRPMovementHeartRateStateType);
```

CRPMovementHeartRateStateType:

MOVEMENT_PAUSE	MOVEMENT_CONTINUE	MOVEMENT_COMPLETE
pause	continue	complete

3. Monitor training state

The training state can also be modified on the bracelet, and the measurement state can be obtained in time through CRPMovementStateListener.onMeasureState().

```
bleConnection.setMovementStateListener(CRPMovementStateListener);
```

4. Set training listener

```
bleConnection.setTrainingListener(CRPTrainingChangeListener listener);
```

5. Query History Training

Query the training records stored in the watch. The query results will be delivered through CRPTrainingChangeListener.onHistoryTrainingChange(List list).

```
bleConnection.queryHistoryTraining();
```

6. Query Training Detail

Query the detail data of a certain training. The query results will be delivered through CRPTrainingChangeListener.onTrainingChange(CRPTrainingInfo info).

```
// onHistoryTrainingChange(List<CRPHistoryTrainingInfo> list)
// id is the index of the list.
bleConnection.queryTraining(int id);
```

2.39 Protocol version

1. Gets the protocol version

The current protocol version can be divided into V1 and V2.

```
CRPProtocolVersion version = bleConnection.getProtocolVersion();
```

2.40 Temperature

1. Start measuring once temperature

After starting the measurement, the real-time temperature is called back through CRPTempChangeListener.onMeasureTemp(), and the measurement state is called back through CRPTempChangeListener.onMeasureTempState().

```
bleConnection.startMeasureTemp();
```

2. Stop measuring once temperature

```
bleConnection.stopMeasureTemp();
```

3. Enable timing temperature measurement

After the timing measurement is turned on, the bracelet automatically measures temperature every half an hour.

```
bleConnection.enableTimingMeasureTemp();
```

4. Disable timing temperature measurement

```
bleConnection.disableTimingMeasureTemp();
```

5. Query the timing of temperature measurement status

The measurement state is called back through `CRPTimingMeasureTempStateCallback.onMeasureState()`.

```
bleConnection.queryTimingMeasureTempState(CRPTimingMeasureTempStateCallback);
```

6. Query the result of timing temperature measurement

The bracelet can save the last two days of timing temperature measurement data, and the result is called back through `CRPTempChangeListener.onTimingMeasureTemp(CRPTempInfo)`.

```
bleConnection.queryTimingMeasureTemp(CRPTempTimeType);
```

`CRPTempTimeType`:

TODAY	YESTERDAY
today	yesterday

`CRPTempInfo`:

type	measureInterval	tempList
CRPTempTimeType	Measurement interval (unit: minute, default 30 minutes)	temperature data

7. Sets listener of temperature measurement results

```
bleConnection.setTempChangeListener(CRPTempChangeListener);
```

8. Enable continuous body temperature measure

```
bleConnection.enableContinueTemp();
```

9. Disable continuous body temperature measure

```
bleConnection.disableContinueTemp();
```

10. Query continuous body temperature measure state

Callback via `CRPTempChangeListener.onContinueState()`.

```
bleConnection.queryContinueTempState();
```

11. 查询最近 24 小时的体温测量记录

通过 `CRPTempChangeListener.onContinueTemp()` 回调。

```
bleConnection.queryLast24HourTemp();
```

2.41 Display time

1. Sets display time

time is the screen-on time, unit: second, cannot exceed 255.

```
bleConnection.sendDisplayTime(time);
```

2. Query display time

The result is called back by `CRPDeviceDisplayTimeCallback.onDisplayTime()`.

```
bleConnection.queryDisplayTime(CRPDeviceDisplayTimeCallback);
```

2.42 Hand washing reminder

1. Set hand washing reminder

Set the start time of hand washing, switch and other information.

```
bleConnection.sendHandWashingReminder(CRPHandWashingPeriodInfo);
```

`CRPHandWashingPeriodInfo`:

enable	startHour	startMinute	count	period
enable	hours	minutes	reminders	reminder interval

2. Query hand washing reminder

The result is called back through `CRPDeviceHandWashingPeriodCallback.onHandWashingPeriod(CRPHandWashingPeriodInfo)`.

```
bleConnection.queryHandWashingReminderPeriod(CRPDeviceHandWashingPeriodCallback);
```

2.43 Sets local city

```
bleConnection.sendLocalCity(city);
```

2.44 Temperature system

1. Sets temperature system

Switch the temperature system of the bracelet.

```
bleConnection.sendTempUnit(CRPTempUnit);
```

CRPTempUnit:

CELSIUS	FAHRENHEIT
Celsius	Fahrenheit

2. Query temperature system

The result is called back by CRPWeatherChangeListener.onTempUnitChange().

```
bleConnection.queryTempUnit();
```

2.45 Brightness

1. Set brightness

```
bleConnection.sendBrightness(int brightness);
```

2. Set brightness

Call back through CRPDeviceBrightnessCallback.onBrightness(int cuurent, int max);

```
bleConnection.queryBrightness(CRPDeviceBrightnessCallback callback);
```

2.46 Classic Bluetooth binding and unbinding

1. Query classic bluetooth Mac address

```
bleConnection.queryBtAddress(CRPBtAddressCallback callback);
```

2. Create Bond

```
BluetoothDevice bluetoothDevice = CRPBleClient.create(Context).getBleDevice(address).getBluetoothDevice();
CRPBtBluetoothBondManager bondManager = CRPBtBluetoothBondManager.getInstance();
bondManager.setBluetoothBondStateListener(new CRPBtBluetoothBondManager.BluetoothBondStateListener() {
    @Override
    public void onBleDisconnect() {
        // Disconnect and close Gatt
    }

    @Override
    public void onBondStateChanged(BluetoothDevice bluetoothDevice, int state) {
        if(state == BluetoothDevice.BOND_BONDED) {
            // Bt bonded
        } else if (state == BluetoothDevice.BOND_NONE) {
            // Bt Unbind
        }
    }

    @Override
    public void onConnectionState(BluetoothDevice bluetoothDevice, int profile, int state) {
        // profile: BluetoothBondStateListener.INDICATOR_A2DP and
        BluetoothBondStateListener.INDICATOR_HEADSET
        // state: BluetoothAdapter.STATE_DISCONNECTED and BluetoothAdapter.STATE_CONNECTED
    }
});
bondManager.createBond(bluetoothDevice);
```

3. Remove Bond

1. Delete BT pairing information

Notify the watch to delete BT pairing information.

```
bleConnection.removeBond();
```

2. Disconnect HEADSET and A2DP

When the BT Mac address is the same as the BLE Mac address, it means dual-mode, and if they are inconsistent, it means single-mode.

```
boolean dualMode = TextUtils.equals(btAddress, bleAddress);  
bondManager.removeBond(blueToothDevice, dualMode);
```

3. Disconnect Gatt

Disconnect and close Gatt (bleDevice.disconnect()) after receiving
CRPBluetoothBondManager.BluetoothBondStateListener.onBleDisconnect().

4. unBond Device

After disconnecting and turning off Gatt, unpair the system.

```
bondManager.unBondDevice();
```

2.47 Contacts

1. Check support contacts

Call back through CRPContactConfigCallback.onContactConfig(CRPContactConfigInfo info);

```
bleConnection.checkSupportQuickContact(CRPContactConfigCallback callback);
```

CRPContactConfigInfo:

supported	count	width	height
Whether to support quick contacts	The count of contacts	The contact avatar width	The contact avatar height

2. Query current contacts count

Call back through CRPContactCountCallback.onContactCount(int count);

```
bleConnection.queryContactCount(CRPContactCountCallback callback);
```

3. Set contacts listener

The setting result is called back through CRPContactListener.

```
bleConnection.setContactListener(CRPContactListener listener);
```

4. Set contacts

CRPContactInfo:

id	width	height	address	name	number
The contacts id	The contacts avatar width	The contacts avatar height	The contacts avatar storage address	The contacts name	The contacts phone number

```
bleConnection.sendContact(CRPContactInfo info);
```

5. Set contacts avatar

The transfer progress is called back through CRPFileTransListener.

```
bleConnection.sendContactAvatar(int id, Bitmap bitmap, int timeout, CRPFileTransListener listener);
```

6. Delete contacts

```
bleConnection.deleteContact(int id);
```

7. Delete contacts avatar

```
bleConnection.deleteContactAvatar(int id);
```

8. Delete all contacts

```
bleConnection.clearContact();
```

9. Query whether the watch supports special symbols

Whether the three symbols +*# are supported in the phone number.

```
bleConnection.queryContactNumberSymbol(CRPContactNumberSymbolCallback);
```

2.48 Battery Saving

1. Set battery saving listener

```
bleConnection.setBatterySavingListener(CRPBatterySavingChangeListener listener);
```

2. Set battery saving state

After the battery saving mode is turned on, the quick view will be turned off.

```
bleConnection.sendBatterySaving(boolean enable);
```

3. Get battery saving state

Call back through CRPBatterySavingChangeListener.onBatterySaving(boolean enable);

```
bleConnection.queryBatterySaving();
```

2.49 Pill Reminder

1. Query support pill reminder

Call back through CRPPillReminderCallback.onPillReminder(int supportCount, List list);

CRPPillReminderInfo:

id	dateOffset	name	repeat	reminderTimeList
The pill id	Start taking medicine in a few days	The pill name	The take medicine every few days	The time point and dosage of the medicine each day

CRPPillReminderInfo.ReminderTimeBean:

time	count
The time to take medicine(For example, 100 is 01:40 am)	The dose

```
bleConnection.queryPillReminder(CRPPillReminderCallback callback);
```

2. Set pill reminder

```
bleConnection.sendPillReminder(CRPPillReminderInfo info);
```

3. Delete pill reminder

```
// CRPPillReminderInfo.id
bleConnection.deletePillReminder(int id);
```

4. Clear pill reminder

```
bleConnection.clearPillReminder();
```

2.50 Tap to wake

1. Query tap to wake state

Call back through CRPTapToWakeCallback.onWakeState(boolean enable);

```
bleConnection.queryTapToWakeState(CRPTapToWakeCallback callback);
```

2. Set tap to wake state

```
bleConnection.sendTapToWakeState(boolean enable);
```

2.51 Create Bond

Some watches require the user to confirm the pairing on the watch after the connection is successful.

```
// After the pairing is completed, you need to save the paired key (bytes), and send the same key after connecting again, without confirming again.
bleConnection.createBond(byte[] bytes, CRPDeviceBondStateCallback callback);
```

2.52 HRV

1. Set HRV listener

```
bleConnection.setNewHrvListener(CRPNewHrvChangeListener listener);
```

2. Query support HRV measure

```
bleConnection.querySupportNewHrv();
```

3. Start measure once HRV

```
bleConnection.startMeasureNewHrv();
```

4. Stop measure once HRV

```
bleConnection.stopMeasureNewHrv();
```

5. Query HRV History

```
bleConnection.queryHistoryNewHrv();
```

2.53 Stress

1. Set stress listener

```
bleConnection.setStressListener(CRPStressListener listener);
```

2. Query support stress measure

If there is a reply, it means that the pressure measurement is supported, and if there is no reply, it means that it does not support it. Via the CRPSressListener.onSupportStress() callback.

```
bleConnection.querySupportStress();
```

3. Start once stress

Callback via CRPSressListener.onStressChange() when the measurement is complete.

```
bleConnection.startMeasureStress();
```

4. Stop once stress

```
bleConnection.stopMeasureStress();
```

5. Query stress history

The watch can save the last 10 measurement records, through the CRPStressListener.onHistoryStressChange() callback.

```
bleConnection.queryHistoryStress();
```

6. Enable timing stress measure

The measurement interval is fixed at half an hour.

```
bleConnection.enableTimingStress();
```

7. Disable timing stress measure

```
bleConnection.disableTimingStress();
```

8. Query timing stress measure status

Call back through CRPStressListener.onTimingStressStateChange().

```
bleConnection.queryTimingStressState();
```

9. Query history timing stress

The watch can save the measurement records of the last two days through the callback of CRPStressListener.onTimingStressChange().

```
bleConnection.queryTimingStress(CRPStressDate);
```

2.54 E-Card

1. Query the count of supported E-Card

If there is a reply, it means that the watch supports E-Card, and if there is no reply, it means that it does not support it. Via the `CRPElectronicCardCountCallback.onElectronicCardCount()` callback.

```
bleConnection.queryElectronicCardCount(CRPElectronicCardCountCallback)
```

CRPElectronicCardCountInfo:

count	urlBytesLimit	savedIdList
The max count supported	url max bytes (utf-8)	List of saved E-Card IDs

2. Set E-Card

```
bleConnection.sendElectronicCard(CRPElectronicCardInfo);
```

CRPElectronicCardInfo:

id	title	url
E-Card id (cannot exceed supported max)	the title	url (the watch uses this url to produce the corresponding QR code)

3. Delete E-Card

```
bleConnection.deleteElectronicCard(id);
```

4. Query E-Card Details

Via the `CRPElectronicCardCallback.onElectronicCard()` callback.

```
bleConnection.queryElectronicCard(id, CRPElectronicCardCallback);
```

5. E-Card Sorting

Reorder E-Cards by idList.

```
bleConnection.sendElectronicCardList(idList);
```

2.55 Calendar Event

1. Set calendar event listener

```
bleConnection.setCalendarEventListener(CRPCalendarEventListener);
```

2. Query support calendar event

The max number of events and saved reminder events are supported through the `CRPCalendarEventListener.onSupportEvent()` callback.

```
bleConnection.querySupportCalendarEvent();
```

3. Set calendar event

```
bleConnection.sendCalendarEvent(CRPCalendarEventInfo);
```

CRPCalendarEventInfo:

id	title	startHour	startMinute	endHour	endMinute	time
The event ID	The event title	Start time(hour)	Start time(minute)	End time(hour)	End time(minute)	reminder time (Timestamp, unit: second)

4. Delete calendar event

```
bleConnection.deleteCalendarEvent(id);
```

5. Query calendar event details

Via the `CRPCalendarEventListener.onEvent()` callback.

```
bleConnection.queryCalendarEvent(int id);
```

6. Set calendar event on status and reminder time

```
// minutes indicates how long to remind in advance, unit: minute, cannot be negative.
bleConnection.sendCalendarEventReminderTime(boolean enable, int minutes);
```

7. Query calendar event on status and reminder time

Via the `CRPCalendarEventListener.onEventReminderTime()` callback.

```
bleConnection.queryCalendarEventReminderTime();
```

8. Delete all calendar event

```
bleConnection.clearCalendarEvent();
```

2.56 GPS

Some watches support GPS positioning, which can be used to record the movement track of outdoor training.

1. Set GPS listener

```
bleConnection.setGpsChangeListener(CRPGpsChangeListener)
```

2. Query history GPS

Callback through CRPGpsChangeListener.onHistoryGpsPathChange(List timeList). timeList is a collection of start time (time stamp, unit: second).

```
bleConnection.queryHistoryGps()
```

3. Query GPS details

Through CRPGpsChangeListener.onGpsPathChange(CRPGpsPathInfo) callback.

```
// time is the timestamp when GPS training started (unit: second)
bleConnection.queryGpsDetail(int time)
```

CRPGpsPathInfo:

time	locationList
the start time	longitude and latitude collection

4. Send epo file

Updating the epo file can help the watch to search stars more quickly. After receiving the CRPGpsChangeListener.onUpdateEpoChange(CRPEpoType) callback, send the corresponding epo file to the watch.

```
bleConnection.sendEpoFile(File, CRPEpoType, CRPFileTransListener)
```

2.58 Vibration Strength

1. Set vibration strength

Adjust the vibration strength of the watch motor.

```
bleConnection.sendVibrationStrength(CRPVibrationStrength strength)
```

2. Query vibration strength

A callback indicates support, and no callback indicates that the watch does not support this function. Callback via CRPVibrationStrengthCallback.onVibrationStrength().

```
bleConnection.queryVibrationStrength(CRPVibrationStrengthCallback callback)
```

2.59 Custom Key

1. **Set listener**

```
bleConnection.setCustomKeyListener(CRPCustomKeyChangeListener listener);
```

2. **Query the list of supported sports types**

Through CRPCustomKeyChangeListener.onSupportTrainingList(List list) callback.

```
bleConnection.querySupportCustomTrainingList();
```

3. **Query the list of supported functions**

Through CRPCustomKeyChangeListener.onSupportFeatureList(CRPCustomKeyInfo info, List list) callback.

```
bleConnection.querySupportCustomFeatureList();
```

4. **Query the list of supported games**

Through CRPCustomKeyChangeListener.onSupportGameList(List list) callback.

```
bleConnection.querySupportCustomGameList();
```

5. Set custom button functions

After successful setting, call back through `CRPCustomKeyChangeListener.onSendResult(boolean success)`.

```
bleConnection.sendCustomKey(CRPCustomKeyInfo info);
```

CRPCustomKeyInfo:

keyType	state	goalType	goalValue
CRPCustomKeyType	0 is disable; 1 is enable	The goal type	The goal value

2.60 World Clock

1. Set listener

```
bleConnection.setWorldClockListener(CRPWorldClockListener listener);
```

2. Check whether world clock is supported

Through `CRPWorldClockListener.onSupportWorldClock(CRPWorldClockInfo info)` callback.

```
bleConnection.querySupportWorldClock();
```

CRPWorldClockInfo:

maxCount	idList
The Max number supported	List of saved clock IDs

3. Query world clock details

Through `CRPWorldClockListener.onWorldClock(CRPWorldClockInfo info)` callback.

```
bleConnection.queryWorldClock(byte id);
```

4. Sent world clock

```
bleConnection.sendWorldClock(CRPWorldClockInfo info);
```

CRPWorldClockInfo:

id	timeZone	longitude	latitude	jetLag	city
The world clock id	Time zone of the city (unit: seconds)	The longitude	The latitude	Time difference from current time zone (unit: seconds)	The city name

5. Delete world clock

```
bleConnection.deleteWorldClock(byte id);
```

6. Reset world clock list

```
bleConnection.resetWorldClockList(byte[] idArray);
```

2.61 Stock

1. Set listener

Stock information needs to be updated after receiving `CRPStockChangeListener.onUpdateStockChange()` callback.

```
bleConnection.setStockListener(CRPStockChangeListener listener);
```

2. Check whether stocks are supported

Through `CRPStockChangeListener.onSupportWorldClock(CRPWorldClockInfo info)` callback.

```
bleConnection.querySupportStock();
```

3. Clear stocks

```
bleConnection.clearStock();
```

4. Send stock

```
bleConnection.sendStock(CRPStockInfo info);
```

CRPStockInfo:

id	abbreviation	code	exchange	currency
The stock id	The stock abbreviation	The stock code	The exchange name	Currency

5. Send stock price

```
bleConnection.sendStockPrice(CRPStockPriceInfo info);
```

CRPStockPriceInfo:

id	enable	price	change	changePercent	openingPrice	highestPrice	lowestPrice	yearHighestPrice	yearLowestPrice	pe	marketValue	turnover	average
The stock id	Close or open the market	Current price	ups and downs	Quote change	Opening price	Today's highest price	Today's low price	Highest price of the year	Lowest price of the year	The pe value	Market value	Volume	Average trading volume

6. ** Delete stock**

```
bleConnection.deleteStock(byte id);
```

7. Reset stock list

```
bleConnection.resetStockList(byte[] idArray);
```

2.62 AI Chat

1. Set listener

```
bleConnection.setChatListener(CRPChatChangeListener listener);
```

2. Send Error Type

```
bleConnection.sendChatError(byte code);
```

3. Start recording

```
bleConnection.sendActivateRecording();
```

4. Send question

After voice recognition, send the question text to the watch.

```
bleConnection.sendChatQuestion(String question);
```

5. Send Response

Send response text to watch.

```
bleConnection.sendChatResponse(String response);
```

2.62 Muslim

1. Send Tasbih Setting

```
bleConnection.sendMuslimTasbihSetting(CRPMuslimTasbihSettingInfo info);
```

CRPMuslimTasbihSettingInfo:

enable	startHour	startMinutes	endHour	endMinutes	interval	repeatMode
Turn on or off	Start time hour (24-hour format)	Start time minutes	End time hours (24-hour format)	End time minutes	Interval time (unit: minutes)	Same CRPAlarmInfo.repeatMode

2. Query Tasbih Setting

Callback through CRPMuslimTasbihSettingCallback.onTasbihSetting(CRPMuslimTasbihSettingInfo info).

```
bleConnection.queryMuslimTasbihSetting(CRPMuslimTasbihSettingCallback callback);
```

3. Set Prayer Setting Listener

在手表端设置或取消，通过 CRPMuslimPrayerSettingListener.onPrayersStateChange(boolean saved, List list) 回调。

```
bleConnection.setMuslimPrayerSettingListener(CRPMuslimPrayerSettingListener listener);
```

4. Save prayers

List of prayer numbers to add to favorites or cancel.

```
bleConnection.sendMuslimPrayersState(boolean saved, List<Integer> list);
```

5. Query saved prayer

Through CRPMuslimPrayerSettingListener.onSavedPrayersChange(List list) callback.

```
bleConnection.queryMuslimSavedPrayers();
```

6. Send prayer notification state

```
// enable is the master switch. When turned off, all notifications will be turned off.
bleConnection.sendMuslimPrayerNotification(boolean enable, Map<CRPMuslimPrayerNotificationType, Boolean>
itemEnableMap);
```

7. Query prayer notification state

Through the CRPMuslimPrayerSettingListener.onNotificationChange() callback.

```
bleConnection.queryMuslimPrayerNotification();
```

8. Send prayer calculation type

```
bleConnection.sendMuslimPrayerCalculationType(CRPMuslimPrayerCalculationType type);
```

9. Query prayer calculation type

Through the CRPMuslimPrayerSettingListener.onPrayerCalculationTypeChange() callback.

```
bleConnection.queryMuslimPrayerCalculationType();
```

10. Set name listener

Set or cancel on the watch side through CRPMuslimNameListener.onNameStateChange(boolean saved, List list) callback.

```
bleConnection.setMuslimNameListener(CRPMuslimNameListener listener);
```

11. Save name

List of names to be saved or canceled.

```
bleConnection.sendMuslimNameState(boolean saved, List<Integer> list);
```

12. Query save name list

Via CRPMuslimNameListener.onSavedNameChange() callback.

```
bleConnection.queryMuslimSavedName();
```

13. Send the current latitude, longitude and time zone

Sent to the watch after receiving the CRPMuslimPrayerSettingListener.updateLocation() callback.

```
bleConnection.sendMuslimLocation(CRPMuslimLocationInfo info);
```

CRPMuslimLocationInfo:

latitude	longitude	timezone
The location latitude	The location longitude	Time zone (unit: minutes)

3 Change log

1.8.2

1. Added Muslim-related interfaces.
2. Fixed occasional CRC error in watch face files.
3. Added message push types.
4. Added AI chat related interfaces.
5. Fixed devices unable to obtain device names on some phones.
6. Adapted to JIELI watch face store and firmware upgrades.

7. Added stock-related interfaces.
8. Added world clock-related interfaces.
9. Added custom button-related interfaces.
10. Added new watch face store interfaces.
11. Fixed occasional null pointer during Realtek ota process.

1.7.1

- Changed

before	1.7.1
syncPastStep	queryHistoryStep
syncPastSleep	queryHistorySleep
syncPastRemSleep	queryHistoryRemSleep
CRPPastTimeType	CRPHistoryDay
CRPDeviceWatchFacesCallback.onWatchFaces()	CRPDeviceWatchFacesCallback.onDisplayWatchFace()
CRPCustomizeWatchFaceInfo	CRPWatchFaceInfo
sendAlarmClock	sendAlarm
queryAllAlarmClock	queryAllAlarm
CRPAlarmClockInfo	CRPAlarmInfo
CRPDeviceAlarmClockCallback.onAlarmClock()	CRPAlarmCallback.onAlarmList()
queryPastHeartRate	queryHistoryTimingHeartRate
enableDrinkWaterReminder and disableDrinkWaterReminder	sendDrinkWaterReminder
CRPDeviceMaximumHeartRateCallback	CRPDeviceMaxHeartRateCallback
enableHandWashingReminder and disableHandWashingReminder	sendHandWashingReminder
CRPWeatherUnit	CRPTempUnit
CRPWeatherChangeListener.onWeatherUnitChange()	CRPWeatherChangeListener.onTempUnitChange()

- Added
 1. Daily goals and training day goals
 2. Sifli download watch face
 3. New alarm
 4. New history training
 5. Bond
 6. HRV
 7. Stress
 8. E-Card
 9. Calendar event
 10. GPS
 11. Vibration strength

1.4.2

1. Changed the settings and query the function name of the sedentary reminder period
2. Changed message push function parameters
3. Added charging state to the bracelet
4. Added metric and imperial switching
5. Added response start or end to find mobile phone
6. Added synchronized music player status and song information
7. Added water reminder
8. Added heart rate alarm
9. Added open training and synchronized training state
10. Added access to protocol version
11. Added temperature measurement
12. Added custom display time
13. Added hand washing system
14. Added local city name
15. Added the switching temperature system

1.3.7

1. Added the physiological cycle related interface.
2. Changed the name of the watch face index.
3. Fixed the Bluetooth connection is not broken after the occasional active disconnection.
4. Added watch UI upgrade.

5. Fixed an issue where you cannot upgrade again after restarting the watch during the upgrade process.
6. After the watch is successfully connected, reset the message queue.
7. Increase the transmission speed of the watch data.
8. Added the watch face background image transmission timeout mechanism.
9. Fixed an exception when the push content was a space.

1.3.4

1. Added the watch face store api.
2. Addes the number of steps statistics api.

1.3.3

1. When the repair file is unpacked, it may be subject to a path traversal attack.

1.3.2

1. Added the firmware upgrade of HS.

1.3.1

1. Cancel the message queue size limit.
2. Changed the shutdown command.

1.3.0

1. Added the transmission of the new dial.
2. Added sports modes such as hiking, tennis, and rugby.

1.2.10

1. Added a list of new version of the watch firmware support language.

1.2.9

1. Added ECG measurement.
2. Added multi-language switching.

1.2.8

1. Fixed abnormal heart rate time in different time zones.

1.2.7

1. Added the watch Italian and Portuguese.

1.2.6

1. Added the lookup phone.

1.2.5

1. Optimize the firmware upgrade process.
2. Added the English firmware change notes.

1.2.4

1. Optimize DFU upgrade speed.
2. Added the setting step size.
3. Added whether the watch is in the DFU data transmission state.

1.2.3

1. A single alarm can set the date, and other types of alarms are not supported.
2. Added a 1.3-inch color screen custom part watchface.